Term Project Final Report

College of Engineering and Computer Science
COMP1020 Object-Oriented Programming & Data Structures
Spring 2026

Team Name: Group 17

Project Title: Chess Plus

Team Members:

Nguyen Quy Tu ID: V202502163
Nguyen Dinh Nam Khanh ID: V202502768
Nguyen Thien Quang ID: V202502841
Bui Anh Minh ID: V202502451
Nguyen Minh Nghia ID: V202502131

1 Introduction and Project Overview

1.1 Problem Statement and Motivation

Chess is a highly complex two-player strategy game with an estimated 10^{120} possible game sequences. A digital chess engine must correctly enforce intricate rules while supporting efficient AI decision-making within practical response times. This makes chess an effective platform for demonstrating Object-Oriented Programming, data structures, and algorithmic problem solving.

1.2 Project Objectives and Scope

The primary objective of *Chess Plus* is to develop a fully functional, rule-complete chess game in Java using a Swing graphical user interface. A secondary objective is to implement AI opponents with progressively advanced search and evaluation techniques. The project supports all standard chess rules including legal movement, check/checkmate/stalemate detection, castling, en passant, and pawn promotion.
Two game modes are supported: Human vs. Human and Human vs. Bot. Four bot difficulty levels are implemented: `BeginnerBot` (depth 3, material evaluation), `AmateurBot` (depth 5, material evaluation), `IntermediateBot` (depth 6, material + Piece-Square Tables), and `HardBot` (depth 6, material + Piece-Square Tables + Quiescence Search).

1.3 Key Features and Functionalities

The system provides an interactive 8×8 graphical chess board with drag-and-drop movement, legal move highlighting, algebraic move logging, and game-over dialogs. Advanced mechanics including castling, en passant, and pawn promotion are fully implemented according to FIDE rules.

To maintain responsiveness during AI search, bot computation executes on a background thread while the UI renders from a temporary board snapshot to prevent flickering. Undo functionality supports both single-player and multiplayer restoration through stored `GameState` snapshots.

1.4 Modifications Since the Interim Stage

At the interim stage, the AI subsystem was incomplete and the UI rendered pieces as text labels. Since then, `BeginnerBot` and `AmateurBot` were implemented using Minimax with Alpha-Beta pruning, while `IntermediateBot` and `HardBot` added Piece-Square Table evaluation and Quiescence Search for tactical stability.
En passant was implemented using `Board.enPassantTarget`. The Undo system was added through the Memento pattern using stacked `GameState` snapshots. The UI was upgraded to PNG-based rendering with a tabular algebraic move log. Bot computation now runs on a background thread; a `pieceSnapshot[][]` is captured before search so `drawPiecesFromSnapshot()` can render a stable board state during calculation. End-game dialogs were wrapped in `SwingUtilities.invokeLater()` to prevent EDT blocking. King positions are cached as `whiteKingSquare` and `blackKingSquare` for $O(1)$ access, and the `ChessBot` interface allows runtime difficulty switching without modifying the UI layer.

2 System Requirements and Specifications

2.1 Core Functionalities

The table below summarises the main system functionalities and their associated classes or methods.

Feature	Description
`isValidMove()`	Each piece class implements its own movement rules
`Board.isChecked()`	Detects attacks on the King using the active piece list
`willMoveResultInCheck()`	Simulates a move to reject illegal positions
`isCheckmate()` `isStalemate()`	Combines check detection with legal move generation
`King.isValidCastle()`	Validates all FIDE castling conditions
`En Passant`	Tracked via `Board.enPassantTarget` and validated in `Pawn.isValidMove()`
`Pawn promotion`	Human players choose a piece via dialog; bots auto-promote to Queen
`BeginnerBot` `AmateurBot`	Minimax + Alpha-Beta at depths 3 and 5 with material evaluation
`IntermediateBot` `HardBot`	Depth-6 Minimax bots using PST evaluation; `HardBot` also uses Quiescence Search
`Stack<GameState>`	Stores deep-copied board snapshots for Undo
`getChessNotation()`	Generates algebraic notation for the move log

2.2 User Requirements and Expected Behaviors

At startup, players select either single-player or multiplayer mode, along with bot difficulty in single-player mode. Pieces respond to drag-and-drop mouse interactions, and invalid moves are automatically rejected. Check states are displayed through `JOptionPane` alerts, while bot moves execute automatically after a short artificial delay to improve gameplay clarity.

2.3 Performance, Usability, and Scalability

To maintain responsiveness during AI search, bot computation executes on a background `Thread`, while UI updates are dispatched through `SwingUtilities.invokeLater()`. Board snapshots are used during search to prevent rendering flicker caused by temporary simulation states.

The system optimizes piece iteration through the `activePieceCoords[]` and `boardToIndex[]` structures, reducing board traversal from $O(64)$ to $O(n)$, where $n \leq 32$. Deeper search depths in `IntermediateBot` and `HardBot` introduce noticeable think times, representing a trade-off for stronger gameplay quality.

3 System Design and Architecture

3.1 High-Level Architecture and Module Organization

The system is divided into backend and frontend layers. The backend manages board state, move validation, game-state detection, turn management, and AI computation, while the frontend handles rendering and mouse interaction. Communication between both layers is coordinated through `GameController`, and the UI never directly modifies the board state.

`ChessBoardUI` acts as the primary view and controller, owning the `Board`, `GameController`, and current `ChessBot` instance. The `Board` manages the `Square[8][8]` grid and active piece indices, while the `Piece` hierarchy encapsulates piece-specific movement logic. Shared utilities such as `MoveHelper` and `Move` support reusable operations throughout the system.

3.2 OOP Concepts, Main Classes, and Interactions

The abstract class `Piece` forms the foundation of the piece hierarchy by storing shared properties such as colour and movement state. It defines the abstract method `isValidMove()`, which each concrete subclass overrides with its own movement rules. This demonstrates abstraction, inheritance, and runtime polymorphism, since the system can validate moves without knowing the concrete piece type in advance.

Each subclass also implements `clonePiece()` for deep-copy snapshot generation during Undo restoration. Encapsulation is applied throughout the design: `Square` stores coordinates, board colour, and a nullable `Piece` reference through controlled access methods, while `Board` wraps the `Square[][]` structure and exposes controlled operations for move execution and simulation.

`GameController` manages turn state and game settings, while `MoveHelper` centralises reusable logic for check detection, move simulation, legal move generation, and checkmate/stalemate testing. The `ChessBot` interface defines a common AI contract, allowing different bot implementations to be swapped at runtime without modifying the UI layer. `Move` encapsulates move-related data, while `GameState` stores board snapshots for the Undo system.

3.3 Class Summary

The system is organised into four major subsystems: core game logic, AI, utility classes, and the user interface. The core logic subsystem consists of `Board`, `Square`, and the `Piece` hierarchy, which manage board state, movement validation, and game-state detection. The AI subsystem is built around the `ChessBot` interface with four difficulty levels. Utility classes support move validation, simulation, and Undo restoration, while the UI subsystem handles rendering and mouse interaction. Table 1 summarises the main classes and interfaces used throughout the system.

Class / Interface	Type	Responsibility
`Piece`	Abstract	Shared piece state and movement contract
`King`, `Queen`, `Rook`, `Bishop`, `Knight`, `Pawn`	Classes	Piece-specific movement and deep-copy logic
`Board`	Class	Board state, simulation, game-state detection, en passant tracking
`Square`	Class	Single board cell with coordinates and piece reference
`GameController`	Class	Turn tracking and game configuration
`ChessBot`	Interface	Defines `getBestMove()`
`BeginnerBot` / `AmateurBot`	Classes	Material-based Minimax + Alpha-Beta bots
`IntermediateBot` / `HardBot`	Classes	Positional Minimax bots using PSTs; `HardBot` adds Quiescence Search
`MoveHelper`	Utility	Check detection and move simulation helpers
`Move`	Data class	Stores move-related data
`GameState`	Inner class	Undo snapshot stored in a `Stack`
`ChessBoardUI`	Class	Main rendering and game-loop controller
`MouseInputListener`	Class	Mouse event translation and drag handling
`TestRunner`	Class	Standalone move-validation tests

Table 1: Main classes and interfaces in the Chess Plus system

4 Data Structures and Algorithms

4.1 Selected Data Structures and Reasons

The chess board is represented using a fixed-size `Square[8][8]` array, providing constant-time access while naturally mapping to chess coordinates. To optimise check detection and move generation, the board maintains an `activePieceCoords[]` list containing only occupied positions, avoiding unnecessary full-board scans. The reverse-mapping array `boardToIndex[]` enables constant-time piece removal during captures.

Undo functionality is implemented through `Stack<GameState>`, which stores deep-copied board snapshots after each move. The AI subsystem uses `List<Move>` to store legal moves during Minimax search, while the UI caches piece images in a `Map<String, BufferedImage>` for efficient rendering.

Direct references to `whiteKingSquare` and `blackKingSquare` allow check detection without repeatedly scanning the board for Kings. `IntermediateBot` and `HardBot` additionally store Piece-Square Tables using `int[][] PIECE_OPENING` and `PIECE_ENDGAME`, enabling $O(1)$ positional lookup per piece.

4.2 Time and Space Complexity

Check detection in `isChecked()` operates in $O(n)$ time, where $n \leq 32$ is the number of active pieces. Legal move generation has worst-case complexity $O(n \times 64)$, since each active piece may evaluate all destination squares. Move simulation in `willMoveResultInCheck()` also runs in $O(n)$ time because the board is temporarily mutated before validation.

The Minimax algorithm has worst-case complexity $O(b^d)$, where b is the branching factor and d is the search depth. Since chess averages roughly 35 legal moves per position, naive Minimax becomes expensive at higher depths. Alpha-Beta pruning significantly reduces evaluated states, approaching $O(b^{d/2})$ in favourable cases.

Active piece removal is optimised to $O(1)$ through the reverse-mapping structure, while Undo restoration requires $O(64)$ time to restore the full board snapshot. Piece-Square Table lookup operates in $O(1)$ per piece using direct 2D array indexing, and PST evaluation remains $O(n)$ because the positional bonus is added during the same traversal as material evaluation.

4.3 Algorithms Implemented

The AI subsystem uses the Minimax algorithm for two-player decision making. White acts as the maximising player while Black acts as the minimising player. The algorithm recursively generates legal moves to a configurable depth, evaluates terminal states using a static evaluation function, and propagates scores back through the game tree.

Alpha-Beta pruning improves search efficiency by maintaining the bounds α and β, pruning branches that cannot influence the final decision. Move ordering further improves pruning performance by prioritising promotions and captures using the Most Valuable Victim / Least Valuable Attacker heuristic.

The evaluation function computes material balance by assigning weighted scores to each piece type. `IntermediateBot` and `HardBot` extend this with Piece-Square Tables, which are 8×8 grids encoding positional principles for each piece type. Each piece has two tables: one for the opening/middlegame and one for the endgame, giving 12 tables in total. Examples include rewarding centralised Knights, rewarding Bishops on long diagonals, rewarding Rooks on the seventh rank, penalising exposed Kings in the opening, and rewarding active Kings in the

endgame. Game phase detection runs in $O(1)$ by checking whether the active piece count is at or below 18. For Black pieces, positional bonuses are mirrored using row $7 - r$.

`IntermediateBot` and `HardBot` also incorporate PST delta into move ordering by adding the positional difference between destination and origin squares before sorting moves. This prioritises moves that improve piece placement and increases Alpha-Beta pruning opportunities.

Quiescence Search is used exclusively by `HardBot` to eliminate the horizon effect, where Minimax stops evaluation immediately before a decisive capture sequence. When the main search reaches depth 0, `quiescenceSearch()` continues exploring captures and promotions until the position becomes tactically stable. A stand-pat score acts as the baseline evaluation, and Alpha-Beta pruning is applied throughout the extended search. This prevents the bot from selecting moves that appear favourable at the search horizon but immediately lose material.

4.4 Trade-offs and Optimisation Decisions

`BeginnerBot` and `AmateurBot` use material-only evaluation to remain fast at lower depths. `IntermediateBot` and `HardBot` extend evaluation with Piece-Square Tables for stronger positional play, trading some speed for improved strategic quality. `HardBot` additionally uses Quiescence Search, which may increase search time considerably in tactical positions.

The active piece list reduces unnecessary board traversal during move generation and check detection. During AI search, the UI renders from a temporary board snapshot while the bot thread evaluates moves in the background, preventing flickering caused by intermediate simulation states.

The Minimax implementation uses a make/undo simulation pattern rather than cloning the entire board at every recursive node. Reusing a single board instance significantly reduces temporary object allocation and avoids the garbage-collection pauses observed in earlier implementations.

4.5 Undo via the Memento Pattern

Undo functionality is implemented using the Memento design pattern. After each committed move, the system stores a complete `GameState` snapshot containing deep copies of the board and pieces.

In single-player mode, Undo restores both the human move and the bot response by popping two states from the stack. In multiplayer mode, only one state is restored per action. Storing complete snapshots simplifies restoration and guarantees correctness even for complex moves such as castling.

5 Implementation Details

5.1 Programming Language, Libraries, and Frameworks

The project is implemented entirely in Java (JDK 11+) without external libraries. The graphical interface uses Java Swing components including `JFrame`, `JPanel`, `JOptionPane`, `JScrollPane`, and `Graphics2D`. Piece images are loaded through `ImageIO` from locally stored PNG assets.

Bot computation executes on a background `Thread`, while UI updates are dispatched through `SwingUtilities.invokeLater()` on the Event Dispatch Thread. The project requires no external build tools and can be compiled directly using `javac` and `java`.

5.2 File Structure and Package Organization

All source files are organised under the `src/lab/` package. Core game management is handled by `Board.java`, `GameController.java`, and `Square.java`, which maintain board state, turn flow, and move validation. The piece hierarchy is built around the abstract `Piece.java` class, with subclasses implementing piece-specific movement behaviour.

Move simulation and Undo support are implemented through `Move.java`, `MoveHelper.java`, and `GameState`. The AI subsystem consists of `ChessBot.java`, `BeginnerBot.java`, `AmateurBot.java`, `IntermediateBot.java`, and `HardBot.java`. User interface rendering and interaction are handled by `ChessBoardUI.java` and `MouseInputListener.java`.

The complete source code is publicly available on GitHub [7].

5.3 Key Implementation Decisions

The `isValidMove()` method is implemented as a pure predicate that validates movement rules without modifying board state. All state mutations are centralised through `Board.applyMove()` and `Board.undoMove()`, allowing AI search and check detection to simulate moves safely without corrupting the live board.

Drag-and-drop interaction is managed through `MouseInputListener`. When a piece is selected, the system computes and highlights legal destination squares. During dragging, the selected piece is rendered at the cursor position, and on release the move is submitted to `tryMove()` for validation and execution.

When the bot's turn begins, `botThinking` is set to true, a snapshot is stored in `pieceSnapshot[][]`, and a background `Thread` runs the Minimax search. While `botThinking` is true, `paintComponent()` renders through `drawPiecesFromSnapshot()` instead of the live board, preventing flickering caused by temporary simulation states. Once the search finishes, `SwingUtilities.invokeLater()` safely applies the selected move on the Event Dispatch Thread.

Castling validation in `King.isValidCastle()` verifies movement history, empty traversal squares, and attacked-square conditions. Pawn promotion is detected during move execution; bots automatically promote to Queen, while human players choose a promotion piece through a dialog.

The Surrender feature reuses the same end-game flow as checkmate and stalemate through `promptNewGame()`, avoiding duplicated logic. AI invocation is also centralised through `triggerBot()`, ensuring consistent behaviour after normal moves and Undo restoration.

En Passant: The target square is tracked globally via `Board.enPassantTarget` and updated only when a pawn moves exactly two squares. During captures, `willMoveResultInCheck()` and `minimax()` carefully orchestrate the removal and restoration of the captured pawn from a different square than the destination.

6 Testing and Evaluation

6.1 Test Cases and Testing Methodology

Testing was performed at both unit and system levels. `TestRunner.java` provides unit-style validation tests by constructing specific board states and evaluating `isValidMove()` outputs against expected results.

The complete system was also tested through manual gameplay sessions covering full match scenarios, including Fool's Mate, Scholar's Mate, stalemate detection, castling, pawn promotion,

Undo restoration, and all AI difficulty levels.

6.2 Backend Validation

Table 2 summarises the primary move validation and gameplay tests.

Table 2: Move validation test results

Test Case	Expected	Status
Pawn forward movement	Valid	Pass
Pawn backward movement	Invalid	Pass
Blocked rook movement	Invalid	Pass
Knight L-shape movement	Valid	Pass
Bishop diagonal movement	Valid	Pass
King moving into check	Rejected	Pass
Valid kingside castling	Allowed	Pass
Castling through check	Rejected	Pass
Checkmate detection	Detected	Pass
Stalemate detection	Detected	Pass
Pawn promotion	Success	Pass
Undo restoration (2-player)	Restored	Pass
Undo restoration (single-player)	Restored	Pass

6.3 Discussion of Correctness, Robustness, and Usability

All standard chess rules within the project scope were validated through the test cases above. The Undo system restores board state safely because each `GameState` stores deep-copied snapshots through `clonePiece()` rather than references to live objects.

The UI remains responsive during AI computation due to the background threading model. Legal move highlighting improves usability by indicating valid destinations, while the algebraic move log provides a complete gameplay history.

6.4 Performance Evaluation

`BeginnerBot` at depth 3 consistently responds in under one second on tested positions. `AmateurBot` at depth 5 typically responds within one to five seconds depending on board complexity. `IntermediateBot` at depth 6 with Piece-Square Tables usually responds within three to ten seconds, while `HardBot` at depth 6 with Piece-Square Tables and Quiescence Search may take five to fifteen seconds or more in tactical positions due to extended capture search at leaf nodes.

No crashes, deadlocks, or invalid game-state transitions were observed during repeated gameplay sessions across all difficulty levels.

7 Challenges and Solutions

7.1 Technical Difficulties

One major challenge involved check detection during move simulation. Early implementations evaluated moves against the live board state rather than the temporary post-simulation state, producing incorrect results during deeper Minimax recursion. This was resolved through a strict apply-check-restore pattern inside `willMoveResultInCheck()`.

Another issue was UI flickering during AI computation. Since Minimax mutates the same board object used by the renderer, temporary simulation states occasionally appeared on screen. The solution was to render from a temporary board snapshot while `botThinking` is active, restoring normal rendering once the AI move is delivered through `SwingUtilities.invokeLater()`.

Castling validation introduced additional edge cases because FIDE rules require simultaneous validation of movement history, path clearance, and attacked squares. These conditions were implemented sequentially inside `King.isValidCastle()` to ensure correctness.

Managing the `activePieceCoords[]` structure also required careful optimisation. Captured pieces are removed using a swap-and-decrement strategy combined with the `boardToIndex[]` reverse map, enabling constant-time removal without shifting the entire array.

The most critical bug encountered was Minimax state corruption caused by ephemeral En Passant target tracking. During recursive simulation, improperly scoped En Passant targets allowed pawns to capture their own colour's ghost pieces. Combined with an uninitialised default in `boardToIndex[]`, this corrupted the `activePieceCoords[]` array, crashed the simulation thread, and triggered false stalemates. The issue was resolved by enforcing strict colour validation in `Pawn.isValidMove()` through `epPawn.isWhite() != this.isWhite()`, and by initialising `boardToIndex[]` to -1 for empty squares to preserve array bounds during deep recursive simulations.

7.2 Design and Architectural Issues

`BeginnerBot` and `AmateurBot` currently share similar Minimax logic and differ mainly in search depth, resulting in partial code duplication. A future improvement would be to replace both classes with a single configurable implementation parameterised by depth.

Another architectural concern is that `ChessBoardUI` still handles both rendering and portions of gameplay flow, creating a risk of excessive responsibility concentration. This was partially mitigated by delegating reusable game-state logic to `MoveHelper` and turn management to `GameController`.

7.3 Lessons Learned

Frequent play-testing proved essential for detecting rule-validation bugs before they became deeply integrated into the codebase. The `ChessBot` interface simplified runtime AI replacement and improved extensibility for future difficulty levels.

The project also reinforced the importance of thread-safe UI programming in Swing applications, since all interface updates must be dispatched through `SwingUtilities.invokeLater()` to avoid race conditions on the Event Dispatch Thread.

8 Conclusion and Future Work

Four AI difficulty levels were implemented: `BeginnerBot` and `AmateurBot` using material-only Minimax, and `IntermediateBot` and `HardBot` extending evaluation with Piece-Square Tables and, for the Hard level, Quiescence Search.

The project demonstrates core Object-Oriented Programming principles through the `Piece` hierarchy, which applies abstraction, inheritance, and runtime polymorphism. Encapsulation is enforced through the `Square` and `Board` classes, while the `ChessBot` interface enables interchangeable AI difficulty levels at runtime. The Memento pattern underpins the Undo system through `GameState` snapshots.

The project also required several architectural and optimisation decisions. Separating backend logic from frontend rendering reduced coupling and simplified development, while background-threaded AI computation maintained UI responsiveness during Minimax search. The active piece index structure further improved move generation and check detection efficiency.

Several limitations remain. Repetition-based draws and the 50-move rule are not currently implemented. In addition, `BeginnerBot` and `AmateurBot` still contain partially duplicated Minimax logic differing mainly in search depth.

Future improvements include implementing the remaining chess rules, refactoring the AI into a configurable `MinimaxBot(depth)` implementation, adding opening-book support, and introducing networked multiplayer and game timer functionality.

9 Team Contributions

Table 3: Team member contributions

Member	Contributions
Nguyen Quy Tu (Leader)	Project coordination; `GameController` and `Board`; check, checkmate, and stalemate detection; active piece index optimisation; Piece-Square Tables.
Nguyen Dinh Nam Khanh	Piece movement validation; sliding-piece path checking; castling validation; pawn promotion logic.
Nguyen Thien Quang	UI development; drag-and-drop interaction; move highlighting; move log panel; bot threading and snapshot rendering; Undo and game controls.
Bui Anh Minh	System integration; `TestRunner`; debugging and integration-stage issue resolution.
Nguyen Minh Nghia	AI subsystem implementation; all four bot classes; Minimax with Alpha-Beta pruning; move ordering; Quiescence Search; Queen logic optimisation.

References

[1] Knuth, D. E., & Moore, R. W. (1975). *An analysis of alpha-beta pruning.* Artificial Intelligence, 6(4), 293–326.

[2] Oracle Corporation. (2024). *Java SE 11 API Documentation.* https://docs.oracle.com/en/java/javase/11/

[3] FIDE. (2024). *FIDE Laws of Chess.* https://www.fide.com/FIDE/handbook/LawsOfChess.pdf

[4] Chessprogramming Wiki. (2024). *Move Ordering.* https://www.chessprogramming.org/Move_Ordering

[5] Chessprogramming Wiki. (2024). *Minimax.* https://www.chessprogramming.org/Minimax

[6] Chessprogramming Wiki. (2024). *Alpha-Beta.* https://www.chessprogramming.org/Alpha-Beta

[7] Group 17. (2026). *Chess Plus - Source Code Repository.* https://github.com/EnTeeKiu/Chess

A Additional Screenshots

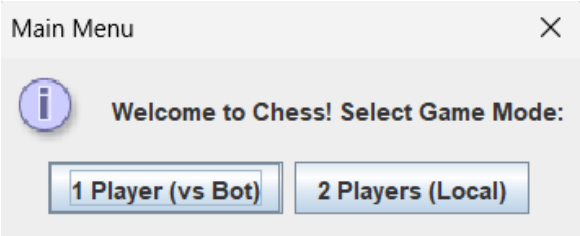

Figure 1: Main menu: game mode selection.

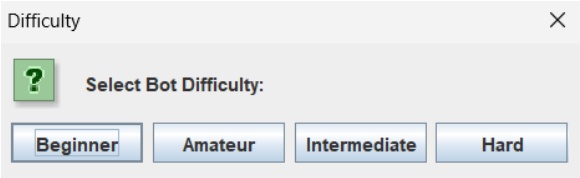

Figure 2: Difficulty selection dialog showing all four bot levels.

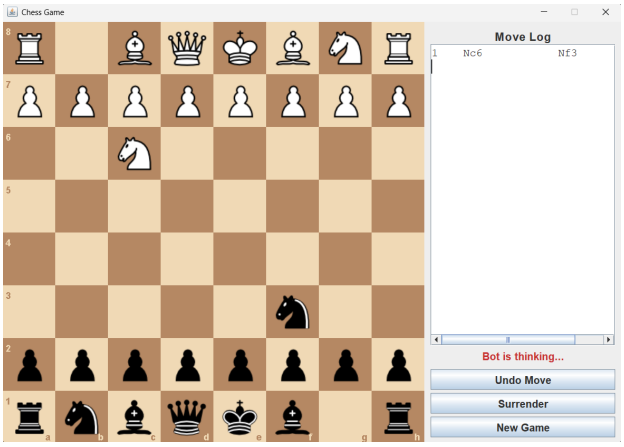

Figure 3: Active game session showing the "Bot is thinking..." indicator, confirming background AI computation keeps the UI responsive.

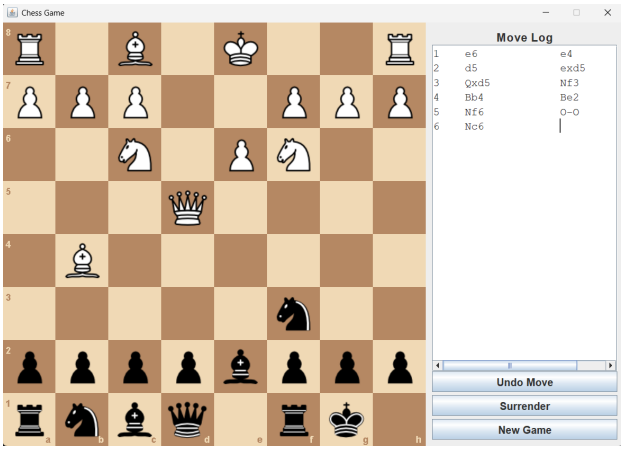

Figure 4: Move log panel recording the complete game history in algebraic notation alongside the Undo Move, Surrender, and New Game controls.

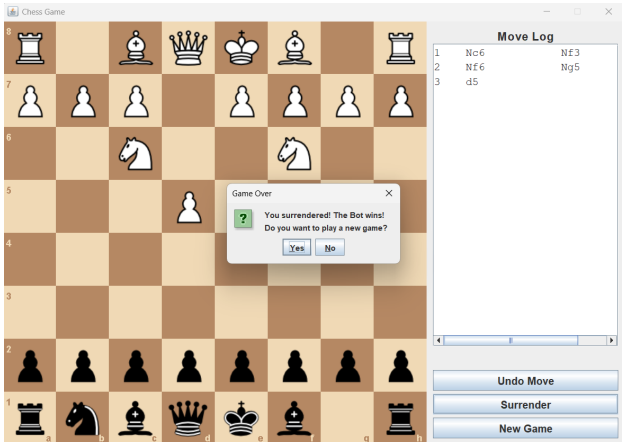

Figure 5: Surrender dialog displayed when the player resigns. The game over prompt offers to start a new game.

Figure 6: Checkmate detected after a 10-move game. The complete move history including pawn promotion is visible in the move log.

B Group Project Task Assignment and Contribution Agreement

Group Members Information

No.	Full Name	Student ID	Email
1	Nguyen Quy Tu (Leader)	V202502163	25tu.nq@vinuni.edu.vn
2	Nguyen Dinh Nam Khanh	V202502768	25khanh.ndn@vinuni.edu.vn
3	Nguyen Thien Quang	V202502841	25quang.nt@vinuni.edu.vn
4	Bui Anh Minh	V202502451	25minh.ba@vinuni.edu.vn
5	Nguyen Minh Nghia	V202502131	25nghia.nm@vinuni.edu.vn

Task Progress and Completion Status

Task	Member	Status	% Done
Core framework and board state	Nguyen Quy Tu	Completed	100%
Piece movement and rule validation	Nguyen Dinh Nam Khanh	Completed	100%
Graphical interface and interaction	Nguyen Thien Quang	Completed	100%
System integration and testing	Bui Anh Minh	Completed	100%
AI subsystem and Minimax search	Nguyen Minh Nghia	Completed	100%

Contribution Agreement

Member Name	Participation	Agree	Comments
Nguyen Quy Tu	100%	Yes	
Nguyen Dinh Nam Khanh	100%	Yes	
Nguyen Thien Quang	100%	Yes	
Bui Anh Minh	100%	Yes	
Nguyen Minh Nghia	100%	Yes	

Declaration

We certify that all contributions and reported responsibilities accurately reflect the work completed throughout the project lifecycle.

Member Name	**Signature**	**Date**
Nguyen Quy Tu	Tu	02/06/2026
Nguyen Dinh Nam Khanh	Khanh	02/06/2026
Nguyen Thien Quang	Quang	02/06/2026
Bui Anh Minh	Minh	02/06/2026
Nguyen Minh Nghia	Nghia	02/06/2026

For TA Use Only:

Approved by: _____ Date: _____